

Security Analysis of Bluetooth Low Energy Communication in IOT Devices

André Guerra^{1*†} and Daniela Dinis^{1*†}

^{1*}School of Technology and Management, Polytechnic University of Leiria,
Leiria, Portugal.

*Corresponding author(s). E-mail(s): andre.r.guerra@ipleiria.pt;
daniela.o.dinis@ipleiria.pt;

[†]These authors contributed equally to this work.

Abstract

Bluetooth Low Energy (BLE) has become a key communication technology for Internet of Things (IoT) devices due to its low power consumption and flexible connectivity. However, its widespread adoption also exposes devices to security threats arising from protocol design choices and constrained implementations. This paper presents a security analysis of BLE communication using ESP32-based IoT devices. Three experimental scenarios are evaluated: an unsecured baseline configuration, an application-layer encryption approach using AES-GCM, and a BLE stack configuration employing LE Secure Connections with authentication and encryption enabled. The scenarios are compared in terms of confidentiality, resistance to eavesdropping, and communication latency. Experimental results show that unsecured BLE communication exposes sensitive data, while both application-layer encryption and LE Secure Connections effectively protect transmitted information, with different impacts on latency. The findings highlight practical security-performance trade-offs and guide for selecting appropriate BLE security mechanisms in resource-constrained IoT deployments.

Keywords: ESP32, Bluetooth, Bluetooth Low Energy, Cybersecurity

1 Introduction

1.1 Context and Motivation

According to the IoT Analytics Research 2025 report [1], the Bluetooth IoT chipset market is expected to grow by 12%. It predicts values from \$7.2 billion in 2024 to \$14.2

billion by 2030. There is continuous interest in integrating Bluetooth Low Energy (BLE) across hospitals, warehouses, and factories.

Bluetooth Low Energy, a subset of Bluetooth 4.0, was introduced in 2010 [2]. While previous Bluetooth Classic (BT) offered point-to-point communication supporting audio transmission and data transfer, BLE adds more diverse communication capabilities [3]. Some of BLE advantages are its extended low power consumption, a versatile range customized to specific needs, and an optimized latency connection [4]. The numerous use cases and advantages of the BLE technology allow us to understand its appeal.

Despite these advantages, BLE remains susceptible to a spectrum of security threats inherent to its protocol design and to resource-limited implementations. BLE security mechanisms such as encryption, authentication, and privacy features involve well-known trade-offs between usability and protection, which can be exploited if poorly implemented [5]. A recent example is WhisperPair [6], a family of attacks that leverages weaknesses in the implementation of the Google Fast Pair protocol, which relies on BLE advertising and discovery mechanisms. In this case, insufficient validation of pairing state and trust assumptions at the accessory level enables unauthorized pairing and device takeover without user interaction.

In this paper, our focus will be the communication between two FireBeetle 2 ESP32-E. The ESP32-E is a ready-to-use microcontroller board with built-in Bluetooth 4.2 classic and BLE and supports programming through the Arduino IDE [7].

1.2 Document Structure

The remainder of this document is organized as follows. Section 2 presents the fundamental concepts related to Bluetooth and Bluetooth Low Energy, including an overview of related work and common attack types targeting BLE communications. Section 3 details the methodology, including the defined test scenarios and evaluation metrics. Section 4 presents the experimental results. Section 5 provides a critical analysis and discussion of the obtained results, highlighting security implications and limitations. Finally, Section 6 concludes the paper and outlines potential directions for future work.

2 Concepts and Related Work

2.1 Bluetooth

Bluetooth is a global short-range wireless communication standard designed to replace wired connections and enable interoperability among heterogeneous devices [8]. The technology operates in the unlicensed 2.4 GHz ISM frequency band and employs the Frequency Hopping Spread Spectrum (FHSS) technique to enhance communication robustness and resilience to interference [4, 8].

There are two main Bluetooth variants that operate independently [4, 9]:

- **Bluetooth Classic (BR/EDR)**: Designed for continuous data transmission and high-bandwidth audio streaming.
- **Bluetooth Low Energy (BLE)**: BLE is optimized for Internet of Things (IoT) applications, focusing on low power consumption through short transmission intervals and deep sleep states, enabling devices to operate for years on small batteries.

Regarding network topologies, Bluetooth supports three fundamental interaction modes [4, 9]:

1. **Point-to-Point:** A direct connection between two devices.
2. **Broadcast:** A single transmitter sends data to multiple receivers without prior connection establishment, commonly used in indoor positioning scenarios.
3. **Mesh:** A many-to-many mesh network that enables large-scale and extended-range systems, suitable for industrial automation and smart city deployments.

2.2 Bluetooth Low Energy (BLE)

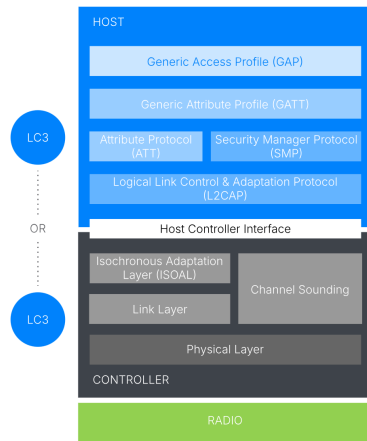


Fig. 1 The Bluetooth LE stack [10]

The BLE protocol architecture is organized as a layered stack, designed to separate radio operations, protocol control, and application functionality while maintaining minimal overhead. At the lowest level, the Physical (PHY) layer and Link Layer (LL) define the radio transmission characteristics, channel hopping, packet framing, and basic encryption procedures. Above this, the Host layer includes the Logical Link Control and Adaptation Protocol (LLCAP), which provides multiplexing and packet segmentation, and the Attribute Protocol (ATT), which governs how data is structured and accessed. BLE data is exposed through the Generic Attribute Profile (GATT), where services and characteristics define device capabilities, while the Generic Access Profile (GAP) manages device discovery, advertising, and connection establishment.

2.3 LE Secure Connections

LE Secure Connections is a BLE security feature introduced in Bluetooth 4.2. Its goal is to make BLE connections much more resistant to eavesdropping and man-in-the-middle (MITM) attacks compared with older legacy methods. The key idea is that, rather than using simple temporary keys, LE Secure Connections employs Elliptic-Curve Diffie-Hellman (ECDH) cryptography: each device generates a private-public key pair, and they exchange public keys to compute a shared secret. This shared secret is then used to create a long-term encryption key that secures the connection. LE Secure Connections supports several pairing methods depending on device capabilities and desired security level, such as “Just Works,” “Passkey Entry,” “Out-of-Band,” and “Numeric Comparison,” with methods involving authenticated user interaction providing better protection against MITM attacks. In practice, this means connections using LE Secure Connections are significantly stronger against passive listening and, when paired with an authenticated method, against active attacks as well, making it the default security mode in modern Bluetooth LE devices.

2.4 Types of BLE attacks

There are several types of BLE attacks. In this section, we mention the main ones. For this paper, we only tested the passive sniffing (Section 2.4.2) as a benchmark.

2.4.1 Denial Of Service (DoS) and Energy Depletion Attacks

Denial of Service and Energy Depletion attacks aim to disrupt device operation by exhausting computational, communication, or energy resources. In low-power technologies such as BLE, these attacks are especially effective due to the strict energy constraints of IoT devices.

Denial of Sleep (DoSL) targets battery-powered devices by repeatedly triggering connection or reconnection events, preventing them from entering low-power sleep states. This forced continuous activity significantly increases energy consumption and can reduce battery lifetime by up to 97%, leading to premature device failure. [3, 5]

Jamming attacks interfere with the wireless channel by transmitting signals that corrupt or block legitimate communications. These attacks may be continuous, random, or reactive, resulting in packet loss, repeated retransmissions, increased energy consumption, and in severe cases complete communication unavailability. [3, 5]

2.4.2 Interception and Active/Passive Eavesdropping

Interception attacks focus on observing or manipulating wireless communications to obtain sensitive information or influence system behavior.

Passive Sniffing (Passive Eavesdropping): The attacker captures radio packets to extract sensitive data. Due to BLE’s predictable frequency hopping pattern, an adversary with appropriate hardware, such as Nordic nRF52840, can follow an active connection and capture traffic when encryption is weak or not enabled. [5]

Man-in-the-Middle (MitM): In this attack, the adversary positions itself between the central and peripheral devices, intercepting and potentially modifying exchanged data. MitM attacks are particularly feasible when devices rely on the “Just Works” pairing method, which provides no protection against such interception. [3, 5]

Replay Attacks: Replay attacks involve capturing legitimate packets, such as a command to unlock a smart lock, and retransmitting them at a later time to trigger the same action without proper authorization. [5, 11]

2.4.3 Implementation Vulnerabilities and Buffer Overflow

Implementation vulnerabilities refer to technical flaws, many of which are documented as CVEs, that can cause device crashes, deadlocks, or forced reboots. In resource-constrained BLE devices, these issues may also lead to serious security compromises.

Link Layer Length Overflow: This vulnerability exploits improper validation of the length field at the BLE link layer. By manipulating this field, an attacker can trigger a buffer overflow in the target device, potentially causing a crash or, in more severe cases, enabling remote code execution. [3, 12, 13]

Zero LTK Installation: This flaw occurs when a device accepts or negotiates a null (all-zero) Long Term Key during the pairing process. As a result, link-layer encryption becomes ineffective, allowing an attacker to bypass security mechanisms entirely and access protected data. [3, 14]

Key Negotiation Downgrade (KNOB) Attack: This attack forces Bluetooth devices to negotiate encryption keys with very low entropy during the key exchange phase. By reducing the key length, the cryptographic strength of the link is weakened, making brute-force key recovery feasible and enabling the attacker to decrypt intercepted communications. [3]

2.4.4 Co-located and Mesh Network Attacks

Co-located and mesh network attacks exploit trust relationships between applications on the same device or routing behavior in multi-hop wireless networks.

Co-located App Attack: This attack occurs when a malicious application running on the same smartphone gains access to pairing credentials, such as the Long Term Key, stored by the operating system. These credentials were originally established by a legitimate application, allowing the malicious app to impersonate trusted devices or access protected BLE communications. [3, 5]

Mesh - Black Hole and Gray Hole Attacks: In mesh networks, a malicious node may deliberately refuse to forward messages. In a Black Hole attack, the node drops all packets, while in a Gray Hole attack, it selectively drops certain messages. Both behaviors disrupt network reliability and compromise data integrity. [3]

Mesh - Trash-can Attack: This attack involves recovering security keys from discarded devices that were not properly wiped before being removed from the mesh network. An attacker can extract residual credentials from device memory and use them to rejoin the network or impersonate legitimate nodes. [3]

2.5 WhisperPair

WhisperPair refers to a family of attacks that exploit weaknesses in the implementation of the Google Fast Pair protocol in Bluetooth accessories, such as wireless earbuds and headphones. Fast Pair is designed to enable quick and seamless Bluetooth pairing with Android devices and relies on Bluetooth Low Energy (BLE) advertisements to discover nearby accessories and initiate the pairing process [15]. However, many accessories fail to correctly enforce the required security checks, in particular the verification that the device is explicitly in pairing mode. [6]

Due to these implementation flaws, an attacker within Bluetooth range can force a pairing without user interaction. This allows the attacker to take control of the accessory in a matter of seconds, including playing unsolicited audio, accessing the microphone, or triggering location exposure through Google’s Find Hub network, when the accessory has not been previously paired with an Android device.

The vulnerability is inherent to the accessory itself and not to the smartphone. As a result, users of non-Android devices, including iPhones, can also be affected if they use vulnerable Bluetooth accessories.

The vulnerabilities were identified by researchers from the COSIC research group at KU Leuven, with the primary authors being Sayon Duttagupta¹ and Seppe Wyls², and were responsibly disclosed to Google. They have been assigned CVE-2025-36911 [16], with a coordinated disclosure process to allow manufacturers to develop and deploy firmware updates. The WhisperPair website³ also provides a list of tested vulnerable

¹<https://www.esat.kuleuven.be/cosic/people/person/?u=u0129899>

²<https://seppe.io/>

³<https://whisperpair.eu/>

devices and recommends keeping accessory firmware up to date as the primary mitigation measure.

3 Methodology

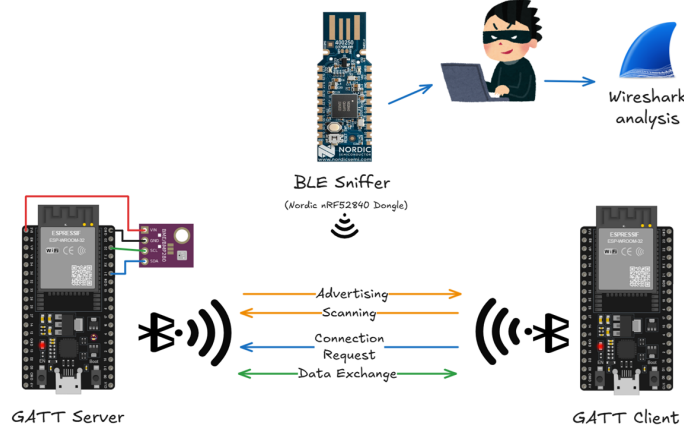


Fig. 2 Architecture with the components used

The methodology followed in this paper systematically aims to analyze the impact of different BLE security mechanisms. This section describes the experimental setup, the roles of the involved devices, and the scenarios used to compare unsecured and secured communication in terms of confidentiality and latency.

The proposed architecture (Figure 2) includes a BLE client and server implemented on ESP32 devices, while a Nordic nRF52840 dongle operates as a passive attacker to capture and inspect wireless communications for security analysis.

3.1 Scenario A

Scenario A is our LE Security Mode 1 - Level 1 implementation. This implementation supposes that there is no security, authentication, or encryption. The intention is to serve as a baseline scenario for subsequent security analysis and comparison with other scenarios. Two ESP32s: one server advertises a BLE service and accepts unencrypted write operations from clients, and one client repeatedly discovers the server, connects, exchanges data, and measures timing metrics. The client also defines the total number of communication rounds. Table 1 show the general flow of behavior of the client and server.

3.2 Scenario B

In this scenario, symmetric cryptography is applied at the application-layer using Advanced Encryption Standard (AES) in Galois/Counter Mode (GCM), providing confidentiality and data authenticity. A shared secret key is assumed to be securely provisioned between the communicating parties prior to establishing the BLE connection.

Table 1 Flow of the function behavior of the server and client

ESP32 BLE Server	ESP32 BLE Client
ESP32 initializes BLE stack and creates server	Creates BLE client
Register BLE service and characteristic using UUIDs	
Starts advertising and waits for client connections ¹	
	Starts a BLE scan for a fixed duration ¹
A BLE connection is established	
	The server target characteristic is located
	Writes data to the characteristic
Reads and logs the received value via serial interface	
	Records the elapsed time for the full scan, connect, and write cycle
	Disconnects from the server
Restarts advertising to become discoverable again ²	Waits briefly before the next iteration ²

¹Beginning of communication round²End of communication round

Based on the implementation of [Scenario A](#), the BLE MTU was increased to 37 bytes to allow the encrypted payload, including nonce and authentication tag, to be transmitted within a single BLE packet. Each encrypted message follows a fixed binary format consisting of a 12-byte GCM nonce with an embedded sequence number, an 8 or 9-byte ciphertext carrying the application data, and a 16-byte authentication tag.

Client-to-server messages have a total payload size of 36 bytes, while server-to-client responses reach 37 bytes to accommodate additional application data, with both fitting within the configured MTU and avoiding fragmentation. Cryptographic operations are performed using a custom library interfacing with mbedTLS, relying on the functions `mbedtls_gcm_crypt_and_tag()` and `mbedtls_gcm_auth_decrypt()`.

3.3 Scenario C

Scenario C implements LE Security Mode 1 - Level 4, representing the highest security level defined by the Bluetooth Low Energy specification. In this scenario, communication is protected using authenticated pairing with LE Secure Connections, ensuring confidentiality, integrity, and protection against Man-in-the-Middle attacks. The BLE server advertises a specific service and enforces encrypted and authenticated access to its GATT characteristic, rejecting any unsecure connections. This scenario focuses on the behavior of the client and server under strict security requirements, serving as a secure reference case for comparison with the unsecured baseline defined in Scenario A.

3.4 Other Testing Scenarios

NimBLE Server-Client: Implements the same scenario described in [Scenario A](#), with the BLE communication stack replaced by the NimBLE library⁴. However, this library could not be used in the experimental evaluation, as the client and server were unable to discover each other during testing, despite being placed only a few centimeters apart.

⁴<https://github.com/h2zero/NimBLE-Arduino>

NimBLE Scanner: Simple BLE scanner that lists all the nearby BLE enabled devices, printing their address, RSSI, and advertised name to the serial console.

Temperature Sketch: Provided practical insight into the correct usage of the BME280 sensor for temperature and pressure measurements, as well as proper configuration and communication over the I²C bus.

4 Results

The following results are based on $20 + 1$ rounds. The extra round is the initial round and is discarded because we have noticed a substantial disparity compared to the other rounds. From our tests, we found the limit to be 31 rounds. When surpassing this number, the client would become unresponsive.

Table 2 presents the qualitative metrics for the three evaluated BLE communication scenarios, including communication confidentiality, authentication and MITM protection, message integrity, and implementation complexity. The table summarizes the security mechanisms applied in each scenario and their respective characteristics.

Table 2 Table of qualitative metrics

Scenario	Communication Confidentiality	Authentication and MITM Protection	Message Integrity	Implementation Complexity
A	No - All communications transmitted in plaintext over BLE	No - No authentication mechanism. Any device can connect and send commands.	No - No integrity protection.	Low - Basic BLE implementation without security features. Simple GATT characteristic read/write operations.
B	Yes - AES-128-GCM encryption at application layer with random 12-byte (96-bit) nonces per message	Partial - Authentication via AES-GCM tags ensures message authenticity and integrity, but NO mutual device authentication (no MITM protection at transport layer)	Yes - Strong integrity protection via AES-GCM authentication tags.	High - Significant effort is required due to extensive payload construction and low-level array manipulation involved in the implementation.
C	Yes - AES-128 encryption via LE Secure Connections	Yes - MITM protection enabled with Passkey Entry + SC pairing.	Yes - Protected by AES-CCM encryption and authentication tags.	Medium-High - Requires correct setup of LE Secure Connections, MITM protection, and bonding procedures.

Table 3 reports the round trip latency measurements obtained during the experimental evaluation. For each scenario, the average, minimum, and maximum latency values are shown, based on the measured communication rounds.

Table 3 Round-trip latency metrics for BLE communication scenarios

Scenario	Avg (ms)	Min (ms)	Max (ms)
A - No Security (Mode 1 Level 1)	27.20	26	27
B - Application-layer encryption	147.45	146	147
C - Mode 1 Level 4 (SC + MITM)	54.45	52	55

5 Analysis and Discussion

The experimental results highlight the security and performance implications of applying different protection mechanisms to BLE communication in IoT devices. By comparing these three scenarios, it is possible to assess how security controls influence both data exposure and communication latency.

Scenario A confirms the inherent insecurity of unprotected BLE communication. As no encryption or authentication mechanisms are enabled, all exchanged data is observable by a passive attacker, validating the effectiveness of simple sniffing attacks in extracting sensitive information. This scenario demonstrates that default or minimal BLE configurations are unsuitable for applications handling confidential or safety-critical data, despite offering minimal latency.

Scenario B demonstrates that application-layer encryption using AES-GCM is effective in preventing data disclosure to passive eavesdroppers. Captured traffic does not reveal plaintext application data, confirming confidentiality and integrity at the message level. However, the results also show a significant increase in round-trip latency compared to the baseline. This overhead is primarily introduced by cryptographic processing and payload construction at the application-layer. Additionally, while message authenticity is ensured, the absence of mutual device authentication means that protection against active man-in-the-middle attacks is not guaranteed.

Scenario C achieves strong protection against both passive and active attacks by leveraging BLE Mode 1 Level 4 with LE Secure Connections and authenticated pairing. In this case, captured traffic remains encrypted at the link-layer, and access to GATT characteristics is restricted to authenticated and encrypted connections. The measured latency is higher than the unsecured scenario but significantly lower than the application-layer encryption approach, indicating that native BLE security mechanisms introduce a moderate but acceptable performance cost.

After setting up the Nordic nRF52840 Dongle, we were able to generate a `.pcap` capture file containing several BLE packets. Using this setup, we attempted to capture the packets exchanged between the client and the server in **Scenario A**, which represents the baseline configuration.

The first issue encountered was related to MAC address identification. The server MAC address displayed in the serial console was `5c:01:3b:39:f3:60`, while in the packet capture it appeared as `5c:01:3b:39:f3:62`. This discrepancy is likely due to the fact that the address printed in the console corresponds to the Wi-Fi interface, whereas Bluetooth uses a different base address. On ESP32 devices, the Bluetooth MAC address is typically offset by +2 from the Wi-Fi MAC address⁵. Based on this behavior, we inferred that the client MAC address would follow the same pattern, namely `34:5f:45:3e:e2:9c` for Wi-Fi and `34:5f:45:3e:e2:9e` for Bluetooth.

⁵https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/misc_system_api.html

Despite this deduction, no packets originating from these Bluetooth MAC addresses were observed during normal client-server operation. The only situation in which packets from the client became visible was when the client was running alone, without the server. Even in this case, only a limited subset of packets could be captured, primarily of the `SCAN_REQ` type.

Repeating the packet sniffing process across different scenarios led to the same outcome. No complete communication exchange between the client and server could be reliably captured. Based on these observations, we conclude that the issue is most likely related to limitations or misconfiguration of the sniffing setup using the nRF52840 dongle, rather than a flaw in the BLE communication itself.

6 Conclusion

The results confirm that unsecured BLE communication, although offering very low latency, is fundamentally unsuitable for applications that handle sensitive or safety-critical data. Passive sniffing alone is sufficient to expose all transmitted information, highlighting the risks of relying on default or minimal BLE configurations. Application-layer encryption effectively mitigates data disclosure and ensures message integrity. However, this approach introduces significant latency and implementation complexity, and it does not inherently protect against active attacks such as Man-in-the-Middle when mutual authentication is absent.

In contrast, the use of LE Secure Connections with authenticated pairing provides robust protection against both passive and active attacks while maintaining a balanced performance profile. Although latency increases compared to the unsecured scenario, it remains substantially lower than that observed with application-layer encryption. This demonstrates that native BLE security mechanisms are not only more secure by design but also more efficient than custom cryptographic solutions implemented at higher layers.

The experimental limitations encountered during packet capture with the nRF52840 dongle further underline the practical challenges of BLE traffic analysis and emphasize the importance of correct tool configuration when performing security assessments. Despite these limitations, the comparative evaluation remains valid and informative.

Overall, the findings reinforce the importance of selecting security mechanisms that align with both the threat model and performance requirements of the target application. For most IoT use cases, LE Secure Connections with authenticated pairing represents the most effective and practical solution. Future work should extend this analysis by considering active attack scenarios, such as Man-in-the-Middle and replay attacks, to evaluate how each BLE security configuration behaves under adversarial interaction. Another relevant direction is the exploration of the vulnerabilities found by the upcoming WhisperPair paper, focusing on implementation weaknesses in BLE pairing mechanisms and their real-world impact.

Additionally, an energy consumption analysis should be conducted to complement the latency results, allowing a clearer understanding of the security-energy trade-offs in resource-constrained IoT devices. Finally, improving the experimental setup to achieve reliable BLE packet sniffing with the nRF52840 dongle is essential, as it would enable deeper traffic analysis and more precise validation of encryption effectiveness across the evaluated scenarios.

References

- [1] Bluetooth IoT Chipset Market: \$14.2 Billion Expected by 2030. <https://iot-analytics.com/bluetooth-iot-chipset-market/> Accessed 2025-11-15
- [2] Liu, C., Zhang, Y., Zhou, H.: A comprehensive study of bluetooth low energy. Journal of Physics: Conference Series **2093**(1), 012021 (2021) <https://doi.org/10.1088/1742-6596/2093/1/012021>
- [3] Lacava, A., Zottola, V., Bonaldo, A., Cuomo, F., Basagni, S.: Securing bluetooth low energy networking: An overview of security procedures and threats. Computer Networks **211**, 108953 (2022) <https://doi.org/10.1016/j.comnet.2022.108953>
- [4] Koulouras, G., Katsoulis, S., Zantalis, F.: Evolution of Bluetooth technology: BLE in the IoT ecosystem. Sensors **25**(4), 996 (2025) <https://doi.org/10.3390/s25040996>
- [5] Barua, A., Al Alamin, M.A., Hossain, M.S., Hossain, E.: Security and privacy threats for bluetooth low energy in iot and wearable devices: A comprehensive survey. IEEE Open Journal of the Communications Society **3**, 251–281 (2022) <https://doi.org/10.1109/ojcoms.2022.3149732>
- [6] Duttagupta, S., Wyns, S., Group, C.R.: WhisperPair: Hijacking Bluetooth Accessories Using Google Fast Pair. <https://whisperpair.eu/> Accessed 2026-01-17
- [7] FireBeetle Board ESP32 E SKU DFR0654. https://wiki.dfrobot.com/FireBeetle_Board_ESP32_E_SKU_DFR0654 Accessed 2025-11-10
- [8] Bisdikian, C.: An overview of the bluetooth wireless technology. IEEE Communications Magazine **39**(12), 86–94 (2001) <https://doi.org/10.1109/35.968817>
- [9] Bluetooth SIG: Bluetooth Technology Overview. <https://www.bluetooth.com/learn-about-bluetooth/tech-overview/> Accessed 2025-11-10
- [10] The Bluetooth® Low Energy Primer — Bluetooth® Technology Website. <https://www.bluetooth.com/bluetooth-le-primer/> Accessed 2025-11-10
- [11] Kaspersky: What Is a Replay Attack? <https://www.kaspersky.com/resource-center/definitions/replay-attack> Accessed 2026-01-20
- [12] MITRE: CVE-2019-16336. <https://www.cve.org/CVERecord?id=CVE-2019-16336> Accessed 2026-01-20
- [13] MITRE: CVE-2019-17519. <https://www.cve.org/CVERecord?id=CVE-2019-17519> Accessed 2026-01-20
- [14] MITRE: CVE-2019-19194. <https://www.cve.org/CVERecord?id=CVE-2019-19194> Accessed 2026-01-20
- [15] Google Developers: Fast Pair Specifications Introduction. <https://developers.google.com/nearby/fast-pair/specifications/introduction> Accessed 2026-01-18

- [16] Mitre / CVE: CVE-2025-36911 Detail. <https://www.cve.org/CVERecord?id=CVE-2025-36911> Accessed 2026-01-18